

Design & Implementation

If I had to try and put my finger on my biggest issues with this, it's that I don't seem to be able to find a structure that makes sense or feels natural. I'm also struggling to put clear boundaries between higher and lower level aspects.

Development environment

Rust was chosen as the primary development language for its performance and safety guarantees while still allowing unsafe, direct memory management if necessary. Its zero-cost abstractions allow higher level intuitive interfaces to be used without causing additional runtime overhead.

Git was used for version control throughout so that branches could be used to develop experimental features in isolation from the main implementation. Additionally this allowed individual experiments and benchmarks to be associated with specific commits for reproducibility.

GitHub was used to host the source code and provide continuous integration through GitHub Actions. Automated unit tests were run on pushes to the default branch, on Windows and Ubuntu environments, providing correctness checks across platforms, ensuring that changes did not introduce subtle logic bugs and or unnacceptably compromise accuracy.

Library Architecture & Implementation

A software library was selected as the project format. This way the simulation can be exposed through a programmatic interface rather than being directly coupled to a particular application or user interface. In future it could be extended with a simulator application that utilises the library while the functionality remains independently accessible.

The library can be organised into three principle layers: the circuit representation, simulation state objects and kernels. The circuit representation describes the operations to be executed, simulation objects maintain simulation state and coordinate execution and kernels implement the mathematical transformations required by operations.

Circuit Representation

This project separates the description of a quantum circuit from the statevector object used to simulate its execution. Quantum operations are represented as instructions defining the operations themselves and contain parameters pertaining to non-implementation specific information. For example rotation gate angles and qubit targeting indices. This results in a clean interface that allows different simulation objects to use different state representations while sharing the same circuit representation, as shown by the statevector and stabiliser implementations.

The core component of this interface is the Instruction enum which provides variants describing different quantum operations, mainly gates. Each variant defines the parameters that may accompany it explicitly, creating a fixed structure for required inputs. The use of an enum also offers a common type the operations can be stored and processed through, making it straightforward to express an entire circuit as an ordered collection of instructions. That ability naturally lends itself to future extensions such as circuit level optimisation as the sequence could be analysed and transformed before execution.

Statevector

A simulation starts by initialising a simulation object such as a statevector, with the desired number of qubits. That statevector then owns the quantum state being simulated using a heap-backed buffer of amplitudes and supplies the interface through which operations can be applied. To progress the simulation an instruction or a circuit as a collection of instructions must be passed to the statevector for execution.

The statevector then acts as a validation and dispatch layer above the operations themselves during execution, while owning the amplitude buffer. By first matching on the instruction the statevector can extract the parameters to perform the necessary validation checks. Validation within the instructions themselves was considered, but checks such as target qubit bounds depend on the state itself. Keeping it in the statevector avoids splitting related checks between the instructions and the statevector object.

After validation, the statevector translates the qubit indices into statevector strides for DPM, its primary simulation strategy, where a stride represents the distance between corresponding amplitudes in the amplitude buffer. This is determined by the position of the target qubit within the chosen big-endian structure. Despite appearing to be a low-level implementation detail, generating the strides in this layer, decouples the kernels that will receive them from the memory layout of the buffer.

The statevector also encapsulates a performance configuration that specifies the hardware features to be utilised during execution. This configuration can be resolved automatically at runtime by detecting the host's CPU capabilities or it can be supplied explicitly to request a specific configuration, which is then validated against available host features. This validation is necessary because executing a kernel that uses unsupported hardware features can result in undefined behaviour.

Qubit measurement is handled directly by the statevector and follows a different execution path from instructions. Unlike unitary operations such as gate application, it returns a value and the circuit representation has no mechanism for collecting results, so measurement does not fit into it naturally. A measurement first calculates the marginal probability of the target qubits outcomes and then samples the outcome by comparing a random number against the probability of it being zero. It then collapses the state by setting amplitudes representative of the false outcome to zero renormalising the remaining amplitudes. Both the probability calculation and the collapse step traverse the amplitude buffer by target stride in the same manner as the DPM kernels. They are currently both implemented directly in the statevector with no kernel dispatch but could be separated into kernels in a future implementation as both could benefit steps could benefit from hardware acceleration.

Kernels

The kernels are separated from the main statevector implementation in a submodule containing four implementations: portable, FMA and AVX. Each performs equivalent operations using different levels of hardware-specific optimisation. The portable implementation provides the DPM kernels and, for reference, the full-system matrix kernels, the FMA implementation provides DPM kernels that use fused multiply-add operations for arithmetic and the AVX implementation provides SIMD single-qubit kernels that adapt DPM but to process multiple amplitude pairs simultaneously. In order to simplify dispatch, all kernels share the same function signatures: a mutable slice of the amplitude buffer, a four element matrix and a target stride calculated by the statevector (controlled two-qubit kernels also require a control stride).

Portable DPM kernels

This is the baseline DPM implementation designed to be the baseline strategy executed with no hardware feature requirements. It can conceptually be split into two categories, functions that traverse the amplitudes and identify pairs of amplitudes and a function that, given these pairs and a matrix, performs the matrix-vector multiplication.

The single-qubit gate kernel consists of an outer loop that walks through the statevector in blocks. Each block contains two halves, each the length of a target stride, corresponding to a different value

of the target qubit. An inner loop then traverses the first half of each block pairing each amplitude with the corresponding amplitude one target stride ahead.

This is especially giving me great difficulty to explain, maybe I can come up with a diagram:

The controlled two-qubit gate kernel adds one further condition: only amplitudes pairs in which the value of the control qubit is one can be updated. The traversal depends on which stride is larger. If the control stride is larger, the first loop steps through the amplitudes in blocks the size of two control strides. Then within the second half of the control block where the control qubit is one, the single-qubit gate traversal can be applied unchanged. If the target stride is larger then the outer loop traverses the amplitudes in blocks the length of two target strides, before a second loop walks through each half where the target qubit is zero and a final inner loop selects amplitudes in sub-blocks where the control is one and pairs each amplitude with another one a target stride ahead.

Matrix application for the kernels is delegated to a dedicated helper function that is called once per amplitude pair. It copies the pairs by value into a buffer on the stack, initialises a second stack buffer to zero for the results, performs the matrix-vector multiplication and then overwrites the original amplitudes with the results. This function can be considered the most performance-critical point in the program as it runs once for every updated pair, so keeping the pair and its results on the stack avoids any execution time penalties due to heap allocations.

FMA DPM kernels

A set of kernels with traversals algorithmically identical to the portable ones but with arithmetic changes to the matrix application. During matrix-vector application fused multiply-add functions are used. These functions only guarantee the result and not the use of hardware FMA instructions. Therefore, the matrix application function is therefore explicitly compiled with hardware FMA instructions using compiler directives. The DPM traversals must be duplicated in this implementation in order to compile them with the same directives since despite not containing FMA instructions it is required for inlining.

AVX DPM kernels

This is especially giving me great difficulty to explain, maybe I can come up with a diagram:

The AVX kernel follows the same traversal as the portable DPM kernel but processes two amplitude pairs per iteration using 256-bit SIMD vectors. Each vector holds four 16-byte double precision floating point values which is enough for two complex amplitudes stored as real and imaginary components. It is compiled with the AVX compiler directive to ensure that the processors SIMD YMM registers are used rather than the vectors being split across two 128-bit registers.

Updating two neighbouring low amplitudes together requires that both are in the first half of the same block, as in order for them to be loaded simultaneously they must be contiguous in memory, so the target stride must be at least two. A target stride of one means that every pair is interleaved in memory, so must be handled by other kernels instead. The outer loop walks the statevector in blocks of two target strides while the inner loop steps through the first half of each block in steps of two. Each iteration therefore updates two consecutive low amplitudes along with their respective partners one target stride ahead.

The matrix is prepared once before the traversal begins, each of the eight values that make up the four complex matrix coefficients are copied across every lane of their own vectors. This is so that a single vector multiplication can apply the same coefficient to both amplitudes held in a register. As the matrix values are invariant across the entire gate application these vectors are created once per

kernel call and passed to the pair routine rather than rebuilding them for every pair. An additional vector containing alternating signs used during complex multiplication is prepared in advance as well.

The main role of the kernel is to multiply a vector of amplitudes by one complex matrix coefficient. This is done by multiplying the amplitude vector by the real coefficient vector, and by the imaginary coefficient vector, both products are then stored separately in their own vectors. To facilitate complex multiplication the lanes of the imaginary products are swapped with their neighbouring lane, before adding the two product vectors and multiplying the result by the sign vector.

The matrix-vector product can then be constructed directly from this operation. The two low amplitudes are packed into a vector and the two high amplitudes into another. The updated low amplitudes are the sum of the top left coefficient applied to the low vector and the top right coefficients applied to the high vector. The updated high amplitudes follow the same process but with the bottom left and right coefficients respectively. Then the vectors are unpacked and written back to the amplitude buffer.

Full-system matrix kernels

Gate application in the full-system matrix kernels acts exactly as described in the Kronecker products background section. They exist as the baseline against which the DPM kernels are measured. They are retained in the portable module but marked as deprecated so that any use outside benchmarks and tests is flagged by the compiler. As the full-system matrix is defined in terms of qubits rather than strides, the kernels first recover the number of qubits using the logarithm of the amplitude count and convert each stride back into a qubit index.

To apply a single-qubit gate the kernel builds the full-system matrix by combining matrix factors using kronecker products across the qubits from first to last. At each position the factor is either the identity matrix, or the gate matrix if the position is the target qubit. The product itself is calculated by a helper that iterates over every factor to apply the next one to the intermediate result. The resulting full-system matrix is then multiplied by the statevector into an output buffer the same length as the state. Only once every amplitude has been computed, the buffer is copied back over the original amplitudes, as overwriting them any earlier would corrupt the inputs for other rows.

The controlled two-qubit kernel follows the projector formulation from the background section but generalises to arbitrary control and target positions. Two full-system matrices are built with the same process as the single-qubit kernel. In the first matrix m_0 the control position receives the projector matrix p_0 , shown in section 2.5, and all other positions receive an identity matrix. This represents the branch where the gate is not triggered. In the second matrix m_1 the control position receives projector matrix p_1 , the target position receives the gate matrix and all other positions again receive the identity matrix. In the same manner as the single-qubit kernel, a helper iterates over these factors and constructs both m_0 and m_1 . Rather than adding the two matrices into a third, the matrix-vector multiplication adds the corresponding elements of both as it applies the matrix to the amplitudes. This avoids a further allocation of the same size but it still requires both matrices to be in memory at once.

Stabiliser

The stabiliser simulator is a small secondary investigation alongside the statevector implementation and applies the Gottesman-Knill theorem introduced in the background chapter. It takes on the same role as the statevector in the library as a simulation object although is not split into a dispatch and kernel layer due to its low complexity.

Its state is a tableau of $2n$ rows where the first n rows are destabilizers and the remaining n rows are stabilizer generators. Each row stores an X bit and a Z bit for every qubit along with a phase bit recording its sign, giving $2n(2n + 1)$ bits in total. While not covered in the background, tracking an equal number of destabilisers provides an alternative measurement method which reduces the time complexity of measurement from x to y .

Measurement for the stabiliser state was not deemed a priority and thus is unimplemented, however destabilisers are tracked in expectation of a future implementation. Despite lacking measurement, the correctness of the stabiliser can be validated against the statevector simulator. The same circuit is run on both simulators starting from the a zero state. The statevector simulator produces the final state while the tableau supplies its n stabiliser generators. Then as the generators represent the actual Pauli gates it's possible to iterate over them and construct a circuit of instructions equivalent to the tableaux internal state. This circuit can then be run on a second zero-initialised statevector and compared to the first.

Testing, benchmarking and observability

Unit tests were used constantly throughout every step of development to ensure the behaviour of the simulator was as expected and inaccuracies were not introduced. Due to the use of floating point arithmetic a certain degree of accumulated numerical error is inevitable. However a tolerance of 10^{-14} was the maximum allowed difference during tests.

For example, using the statevector-stabiliser test described above every circuit of up to three gates on three qubits drawn from the supported gates each stabiliser generator including its sign must leave the state produced by the statevector simulator unchanged. This covers 6175 circuits and tests the phase updates directly.

Criterion.rs, a Rust port of the popular Haskell Criterion package, was used for execution time benchmarking and analysis. It iterates over benchmarks gathering samples and then provides a full statistical breakdown. It also supports direct comparison benchmarks useful for verifying the difference between different approaches and algorithms. This was most useful when it came to developing different gate application algorithms and analysing the effects of compiler directive placement.

DHAT was used to investigate the behaviour of heap allocations. It is registered as the global allocator and wraps the default allocator so that every allocation and deallocation is recorded while profiling is active. From this it can report the total number of allocations and bytes requested alongside peak heap usage. Each allocation also records the backtrace of the call that made it, which aids when analysing what these allocations can be attributed to.

The Tracing crate was used to provide observability insights into circuit execution. Primarily useful when examining sub-routines such as the quantum Fourier transform which expands into a series of gates whose ordering depends on parameters such as qubit count. It provided quality-of-life features such as exporting trace data to Chrome Perfetto for examination.

Rust's feature gating was used extensively throughout in coordination with these features to ensure that any accommodations for benchmarking and tracing did not introduce unnecessary overhead when unactivated.